

Objective:

The main of the purpose of this project was to implement and monitor a secure local web environment using:

- A local domain secured with an SSL/TLS certificate (which was hamza.local)
- Elasticsearch, Logstash and Kibana (ELK) for log management.
- To use the Squid Proxy for web traffic control and monitoring.
- A DMZ firewall configuration using iptables for network segmentation and enhanced security.

1. Local domain Setup and SSL/TLS Certificate:

Purpose: Establish an identity on the web and secure it via X509 certificate.

Steps:

Configuration of Local Domain:

1. Provision local domain entry

`sudo nano /etc/hosts`

```
hamza@hamza:~$ sudo nano /etc/hosts
```

After that, I have used my first name (hamza) as per requirement (hamza.local) in the host file

```
GNU nano 7.2 /etc/hosts
127.0.0.1 hamza.local
::1 localhost6.localdomain6 localhost6

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts

[ Read 9 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

2. Installed apache web server

`sudo apt-get install apache2`

```
hamza@hamza:~$ sudo apt-get install apache2
Reading package lists... Done
```

3. Configured the web server for hamza.local:

```
sudo vim /etc/apache2/sites-available/hamza.local.conf
```

```
<VirtualHost *:80>
```

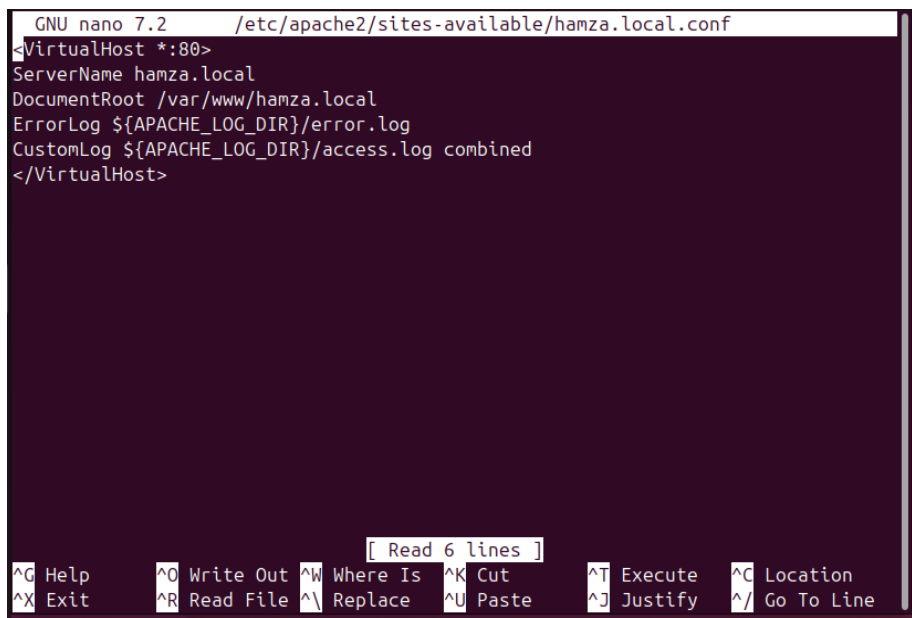
```
ServerName hamza.local
```

```
DocumentRoot /var/www/hamza.local
```

```
ErrorLog ${APACHE_LOG_DIR}/error.log
```

```
CustomLog ${APACHE_LOG_DIR}/access.log combined
```

```
</VirtualHost>
```

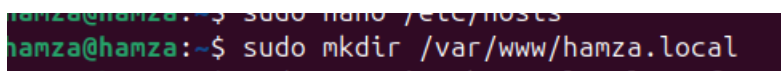
A screenshot of the GNU nano 7.2 text editor editing the file /etc/apache2/sites-available/hamza.local.conf. The editor shows the following configuration: <VirtualHost *:80>, ServerName hamza.local, DocumentRoot /var/www/hamza.local, ErrorLog \${APACHE_LOG_DIR}/error.log, CustomLog \${APACHE_LOG_DIR}/access.log combined, and </VirtualHost>. The status bar at the bottom indicates 'Read 6 lines' and provides various keyboard shortcuts for editing and navigation.

```
GNU nano 7.2 /etc/apache2/sites-available/hamza.local.conf
<VirtualHost *:80>
ServerName hamza.local
DocumentRoot /var/www/hamza.local
ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>

[ Read 6 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

4. Created the root directory for the site:

```
sudo mkdir /var/www/hamza.local
```

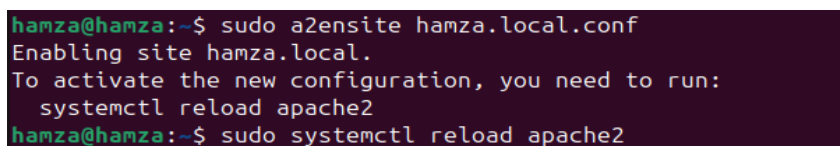
A terminal screenshot showing the command 'sudo mkdir /var/www/hamza.local' being executed successfully. The prompt is hamza@hamza:~\$ and the output is a green message indicating the directory was created.

```
hamza@hamza:~$ sudo mkdir /var/www/hamza.local
mkdir: created directory '/var/www/hamza.local'
```

5. Enabled the site and after that restarted Apache:

```
sudo a2ensite hamza.local.conf
```

```
sudo systemctl reload apache2
```

A terminal screenshot showing the commands 'sudo a2ensite hamza.local.conf' and 'sudo systemctl reload apache2' being executed. The output shows the site being enabled and a message to reload Apache.

```
hamza@hamza:~$ sudo a2ensite hamza.local.conf
Enabling site hamza.local.
To activate the new configuration, you need to run:
  systemctl reload apache2
hamza@hamza:~$ sudo systemctl reload apache2
```

6. Tested the local domain which I have setup (which is `http://hamza.local`)



2. Setup SSL/TLS certificate for the site:

Steps:

1. In the first step we have generated keypair (signed using local CA) and SSL Certificate:

1.1 Generated the keypair (hamza.local.key) and a self-signed certificate

(hamza.local.crt):

```
openssl req -x509 -newkey rsa:4096 -sha256 -days 365 -nodes
```

```
keyout hamza.local.key -out hamza.local.crt -subj
```

```
"/CN=hamza.local" -addext
```

```
"subjectAltName=DNS:hamza.local,DNS:*.hamza.local"
```

[illegible]

1.2: In the next step we have generated the CSR (hamza.local.csr) for the keypair (hamza.local.key):

```
openssl req -new -key hamza.local.key -out hamza.local.csr
```

```
hamza@hamza:~$ openssl req -new -key hamza.local.key -out hamza.local.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:IDF
Locality Name (eg, city) []:Paris
Organization Name (eg, company) [Internet Widgits Pty Ltd]:EPITA
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:hamza.local
Email Address []:admin@local.tld

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:@12345678@
An optional company name []:
```

1.3 Generated local CA keypair (myCA.key):

```
sudo openssl genrsa -des3 -out /etc/ssl/private/myCA.key 2048
```

```
hamza@hamza:~$ sudo openssl genrsa -des3 -out /etc/ssl/private/myCA.key 2048
[sudo] password for hamza:
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
hamza@hamza:~$
```

1.4 Self-sign the local CA public key (myCA.pem) using local CA own keypair (myCA.key):

```
sudo openssl req -x509 -new -nodes -key /etc/ssl/private/myCA.key -sha256 -days 1024 -out myCA.pem
```

```
hamza@hamza:~$ sudo openssl req -x509 -new -nodes -key /etc/ssl/private/myCA.key -sha256 -days 1024 -out myCA.pem
[sudo] password for hamza:
Enter pass phrase for /etc/ssl/private/myCA.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:IDF
Locality Name (eg, city) []:Paris
Organization Name (eg, company) [Internet Widgits Pty Ltd]:EPITA ROOT
Organizational Unit Name (eg, section) []:EPITA ROOT CA
Common Name (e.g. server FQDN or YOUR name) []:root.tld
Email Address []:
```

1.5 Processed the CSR (hamza.local.csr) to sign the site's certificate

(hamza.local.crt) using the local CA (myCA.pem) keypair

(myCA.key). Include the subjectAltName from a separate configuration file (SAN.cnf) to prevent the 'SSL_ERROR_BAD_CERT_DOMAIN' browser SSL/TLS error.

sudo nano SAN.cnf

```
hamza@hamza:~$ sudo nano SAN.cnf
```



```
[ req ]
distinguished_name = req_distinguished_name
req_extensions = v3_req
prompt = no
[ req_distinguished_name ]
CN = salman.local
[ v3_req ]
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names
[ alt_names ]
DNS.1 = hamza.local
DNS.2 = *.hamza.local
```

1.6 I did the verification by using this command

openssl verify -CAfile myCA.pem hamza.local.crt

```
hamza@hamza:~$ openssl verify -CAfile myCA.pem hamza.local.crt
hamza.local.crt: OK
```

2. Moved the site keypair (hamza.local.key) and signed certificate (hamza.local.crt) to the correct paths:

```
hamza@hamza:~$ sudo mv hamza.local.key /etc/ssl/private/hamza.local.key
hamza@hamza:~$ sudo mv hamza.local.crt /etc/ssl/certs/hamza.local.crt
hamza@hamza:~$ sudo mv myCA.pem /etc/ssl/certs/myCA.pem
hamza@hamza:~$
```

3. Set the permissions:

```
hamza@hamza:~$ sudo chmod 600 /etc/ssl/private/hamza.local.key
```

4. Created the Apache SSL Virtual Host Configuration

```
hamza@hamza:~$ sudo nano /etc/apache2/sites-available/hamza.local-ssl.conf
```

```
GNU nano 7.2 /etc/apache2/sites-available/hamza.local-ssl.conf *
<IfModule mod_ssl.c>
<VirtualHost _default_:443>
ServerAdmin webmaster@hamza.local
ServerName hamza.local
DocumentRoot /var/www/hamza.local

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/ssl_access.log combined

SSLEngine on
SSLCertificateFile /etc/ssl/certs/hamza.local.crt
SSLCertificateKeyFile /etc/ssl/private/hamza.local.key
SSLCACertificateFile /etc/ssl/certs/myCA.pem

<FilesMatch "\.(cgi|shtml|phtml|php)$">
SSLOptions +StdEnvVars
</FilesMatch>
<Directory /usr/lib/cgi-bin>
SSLOptions +StdEnvVars
</Directory>
</VirtualHost>
</IfModule>

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy
```

Add this in the hamza.local-ssl.conf

5. Enabled the SSL site:

```
sudo a2ensite hamza.local-ssl.conf
```

6. Enabled the Apache SSL module (or ensure its activation):

```
sudo a2enmod ssl
```

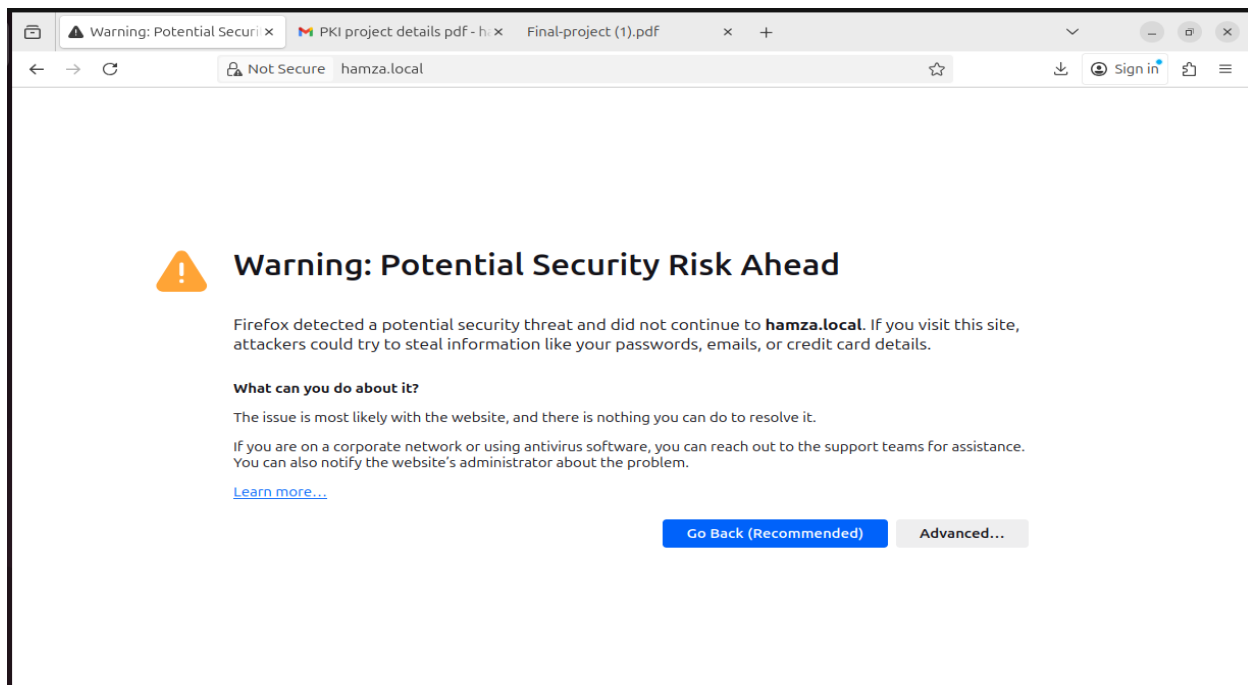
7. Restarted the apache:

```
sudo systemctl restart apache2
```

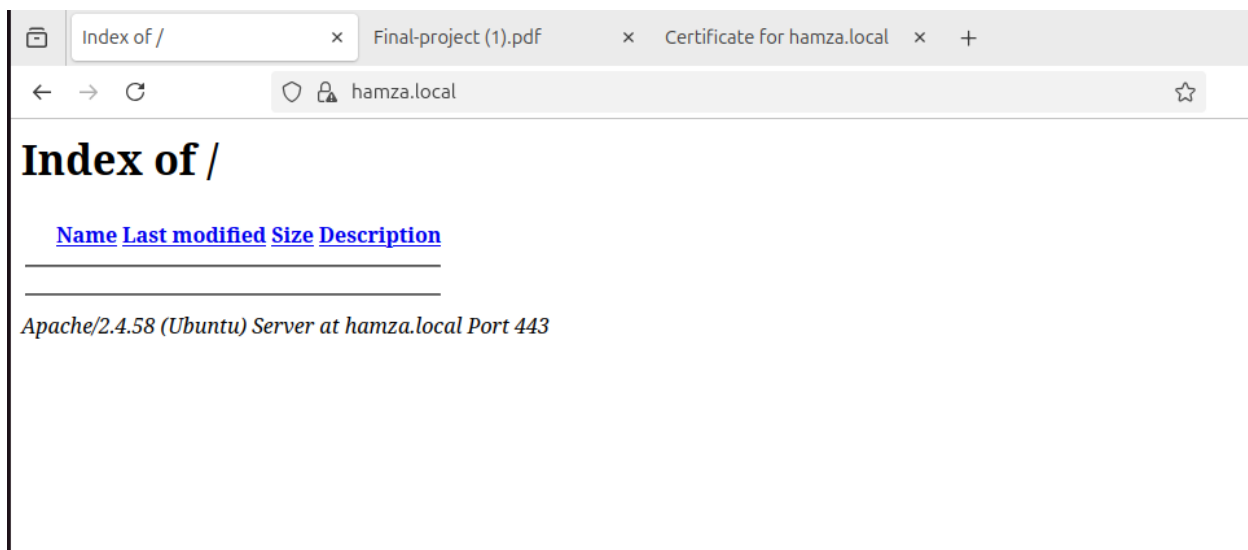
```
hamza@hamza:~$ sudo nano /etc/apache2/sites-available/hamza.local-ssl.conf
hamza@hamza:~$ sudo a2ensite hamza.local-ssl.conf
Enabling site hamza.local-ssl.
To activate the new configuration, you need to run:
    systemctl reload apache2
hamza@hamza:~$ sudo systemctl reload apache2
hamza@hamza:~$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2
hamza@hamza:~$ sudo systemctl restart apache2
```

8. Tested the SSL/TLS setup on local domain

<https://hamza.local>



After that, I went to advanced and trusted this CA and it will show that it is secured now.



9. After securing it, we can see the certificate
Certificate

hamza.local		root.tld	
Subject Name			
Country	FR		
State/Province	IDF		
Locality	Paris		
Organization	EPITA		
Common Name	hamza.local		
Email Address	admin@local.tld		
Issuer Name			
Country	FR		
State/Province	IDF		
Locality	Paris		
Organization	EPITA ROOT		
Organizational Unit	EPITA ROOT CA		
Common Name	root.tld		
Validity			
Not Before	Mon, 10 Nov 2025 00:55:54 GMT		
Not After	Tue, 10 Nov 2026 00:55:54 GMT		
Subject Alt Names			
DNS Name	hamza.local		
DNS Name	*.hamza.local		

Subject Alt Names	
DNS Name	hamza.local
DNS Name	*.hamza.local
Public Key Info	
Algorithm	RSA
Key Size	4096
Exponent	65537
Modulus	93:B5:FE:4A:0C:B7:3E:25:AA:4E:73:60:4B:0C:FB:C1:4D:7D:11:F9:2F:B0:61:35...
Miscellaneous	
Serial Number	3F:3A:5E:11:43:EB:B0:1A:F2:43:F0:27:8C:35:2B:B9:46:79:FF:E4
Signature Algorithm	SHA-256 with RSA Encryption
Version	3
Download	PEM (cert) PEM (chain)
Fingerprints	
SHA-256	A7:CA:9E:D3:03:C1:E2:31:43:83:EE:3B:83:FB:E3:AF:6F:93:34:87:17:14:6E:6C...
SHA-1	27:D0:25:95:5C:A0:80:92:BA:F0:A4:69:82:D3:7C:EF:35:73:26:42
Key Usages	
Purposes	Digital Signature, Key Encipherment
Extended Key Usages	
Purposes	Server Authentication
Subject Key ID	
Key ID	B1:01:C6:15:D0:CE:51:7B:05:1F:B3:C1:DE:FC:A2:65:3A:C2:F9:F3
Authority Key ID	
Key ID	D9:05:E7:C0:70:8D:29:E8:B7:FE:0E:2D:99:8F:12:1E:D2:29:8F:41

2. ELK Stack Implementation and Apache Log Ingestion

1. Installed the Elasticsearch:

1.1 Install required dependencies:

`sudo apt install openjdk-11-jdk wget apt-transport-https curl gpgv gpgsm gnupg-l10n gnupg dirmngr -y`

```
hamza@hamza:~$ sudo apt install openjdk-11-jdk wget apt-transport-https curl gpgv gpgsm gnupg-l10n gnupg dirmngr -y
```

1.2 Added the Elastic components cause they are not present in Ubuntu by default

`wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add -`

`echo "deb https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee -a /etc/apt/sources.list.d/elastic-7.x.list`

```
hamza@hamza:~$ wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add -
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instead (see apt-key(8)).
OK
hamza@hamza:~$
```

```
hamza@hamza:~$ echo "deb https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee -a /etc/apt/sources.list.d/elastic-7.x.list
deb https://artifacts.elastic.co/packages/7.x/apt stable main
```

1.3 updated the package list: `sudo apt update`

```
hamza@hamza:~$ sudo apt update
```

1.4 Install Elasticsearch: `sudo apt install elasticsearch -y`

```
hamza@hamza:~$ sudo apt install elasticsearch -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  elasticsearch
```

1.5 Restricted the listening of traffic by (default) port 9200 (via REST API) to 'localhost'

`sudo nano /etc/elasticsearch/elasticsearch.yml`

```
hamza@hamza:~$ sudo nano /etc/elasticsearch/elasticsearch.yml
```

```
GNU nano 7.2 /etc/elasticsearch/elasticsearch.yml
# on the system and that the owner of the process is allowed to use this
# limit.
#
# Elasticsearch performs poorly when the system is swapping the memory.
#
# ----- Network -----
#
# By default Elasticsearch is only accessible on localhost. Set a different
# address here to expose this node on the network:
#
network.host: localhost
#
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
#
#http.port: 9200
#
# For more information, consult the network module documentation.
#
# ----- Discovery -----
#
# Pass an initial list of hosts to perform discovery when this node is started:
# The default list of hosts is ["127.0.0.1", "[::1]"]
#
#discovery.seed_hosts: ["host1", "host2"]
#
# Bootstrap the cluster using an initial set of master-eligible nodes:
#
^C Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^/_ Go To Line M-E Redo     M-6 Copy
```

1.6 Start Elasticsearch service (logs: /var/log/elasticsearch): `sudo systemctl start elasticsearch`

1.7: Verified via: `curl -X GET localhost:9200`

```
hamza@hamza:~$ curl -X GET localhost:9200
{
  "name" : "hamza",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "sVq9Ie2JToyMgP0LfBpb5Q",
  "version" : {
    "number" : "7.17.29",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "580aff1a0064ce4c93293aaab6fcc55e22c10d1c",
    "build_date" : "2025-06-19T01:37:57.847711500Z",
    "build_snapshot" : false,
    "lucene_version" : "8.11.3",
    "minimum_wire_compatibility_version" : "6.8.0",
    "minimum_index_compatibility_version" : "6.0.0-beta1"
  },
  "tagline" : "You Know, for Search"
}
```

2. Install the Kibana:

2.1 Installed the Kibana by using this command: `sudo apt install kibana`

2.2 Started it by using this command : `sudo systemctl start kibana`

2.3 Verified the status that it is running by using: `sudo systemctl status kibana`

```

hamza@hamza:~$ sudo systemctl start kibana
hamza@hamza:~$ sudo systemctl status kibana
● kibana.service - Kibana
   Loaded: loaded (/etc/systemd/system/kibana.service; disabled; preset: enabled)
   Active: active (running) since Mon 2025-11-10 03:02:19 CET; 17s ago
     Docs: https://www.elastic.co
  Main PID: 13151 (node)
    Tasks: 11 (limit: 10477)
   Memory: 201.8M (peak: 201.9M)
      CPU: 13.120s
   CGroup: /system.slice/kibana.service
           └─13151 /usr/share/kibana/bin/./node/bin/node /usr/share/kibana/bin/./src/cli/dist --logging.dest=/var/log/kibana.log

Nov 10 03:02:19 hamza systemd[1]: Started kibana.service - Kibana.
Nov 10 03:02:20 hamza kibana[13151]: Kibana is currently running with legacy OpenSSL providers enabled! For details and
lines 1-13/13 (END)

```

As we can see, the status is “Active”

3. Installing logstash

installed logstash using this command: `sudo apt install logstash`

```

hamza@hamza:~$ sudo apt install logstash

```

4. Configure Logstash to Process Apache Logs:

Create a new Logstash pipeline:

`sudo nano /etc/logstash/conf.d/apache.conf`

```

hamza@hamza:~$ sudo nano /etc/logstash/conf.d/apache.conf

```

Configured grok filter to parse logs and output to Elasticsearch.

```

hamza@hamza:~$ sudo nano /etc/logstash/conf.d/apache.conf
GNU nano 7.2 /etc/logstash/conf.d/apache.conf
input {
  file {
    path => "/var/log/apache2/access.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
}
filter {
  if [path] =~ "access" {
    mutate { replace => { "type" => "apache_access" } }
    grok {
      match => { "message" => "%{COMBINEDAPACHELOG}" }
    }
  }
  date {
    match => [ "timestamp", "dd/MMM/yyyy:HH:mm:ss Z" ]
  }
}
output {
  elasticsearch {
    hosts => ["localhost:9200"]
  }
}

```

Read 23 lines

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
 ^X Exit ^R Read File ^_ Replace ^U Paste ^D Justify ^_ Go To Line M-E Redo M-G Copy

5. Ensured that all services are enabled by this command:

sudo systemctl status

```
hamza@hamza:~$ sudo systemctl status
● hamza
   State: running
  Units: 559 loaded (incl. loaded aliases)
   Jobs: 0 queued
 Failed: 0 units
  Since: Sun 2025-11-09 23:44:14 CET; 4h 50min ago
systemd: 255.4-1ubuntu8.10
 CGroup: /
        └─init.scope
             └─1 /sbin/init splash
        └─system.slice
             ├─ModemManager.service
             │   └─892 /usr/sbin/ModemManager
             ├─NetworkManager.service
             │   └─822 /usr/sbin/NetworkManager --no-daemon
             ├─accounts-daemon.service
             │   └─747 /usr/libexec/accounts-daemon
             ├─apache-htcacheclean.service
             │   └─4752 /usr/bin/htcacheclean -d 120 -p /var/cache/apache2/mod_cache_disk -l 300M -n
             ├─apache2.service
             │   ├──9941 /usr/sbin/apache2 -k start
             │   ├──9943 /usr/sbin/apache2 -k start
             │   └─9944 /usr/sbin/apache2 -k start
             ├─avahi-daemon.service
             │   ├──705 "avahi-daemon: running [hamza.local]"
             │   └─819 "avahi-daemon: chroot helper"
             ├─colord.service
             │   └─1401 /usr/libexec/colord
             └─cron.service
```

And after that checked individually that logstash, kibana and elasticsearch services are running

```
hamza@hamza:~$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; disabled; preset: enabled)
   Active: active (running) since Mon 2025-11-10 05:53:30 CET; 4min 57s ago
     Docs: https://www.elastic.co
   Main PID: 5236 (java)
    Tasks: 84 (limit: 10477)
   Memory: 4.8G (peak: 4.9G)
      CPU: 3min 5.009s
   CGroup: /system.slice/elasticsearch.service
           └─5236 /usr/share/elasticsearch/jdk/bin/java -Xshare:auto -Des.networkaddress.cache.ttl=60 -Des.networkaddr...
           └─5431 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Nov 10 05:52:44 hamza systemd[1]: Starting elasticsearch.service - Elasticsearch...
Nov 10 05:53:03 hamza systemd-entrypoint[5236]: Nov 10, 2025 5:53:03 AM sun.util.locale.provider.LocaleProviderAdapter
Nov 10 05:53:03 hamza systemd-entrypoint[5236]: WARNING: COMPAT locale provider will be removed in a future release
Nov 10 05:53:30 hamza systemd[1]: Started elasticsearch.service - Elasticsearch.
lines 1-16/16 (END)
```

```
hamza@hamza:~$ sudo systemctl status logstash
● logstash.service - logstash
   Loaded: loaded (/etc/systemd/system/logstash.service; disabled; preset: enabled)
   Active: active (running) since Mon 2025-11-10 05:46:06 CET; 13min ago
   Main PID: 4365 (java)
    Tasks: 42 (limit: 10477)
   Memory: 779.6M (peak: 852.1M)
      CPU: 3min 3.972s
   CGroup: /system.slice/logstash.service
           └─4365 /usr/share/logstash/jdk/bin/java -Xms1g -Xmx1g -XX:+UseConcMarkSweepGC -XX:CMSInitiatingOccupancyFr...

Nov 10 05:53:18 hamza logstash[4365]: [2025-11-10T05:53:18,775][INFO ][logstash.outputs.elasticsearch][main] Failed to
Nov 10 05:53:18 hamza logstash[4365]: [2025-11-10T05:53:18,778][WARN ][logstash.outputs.elasticsearch][main] Attempted
Nov 10 05:53:23 hamza logstash[4365]: [2025-11-10T05:53:23,852][INFO ][logstash.outputs.elasticsearch][main] Failed to
Nov 10 05:53:23 hamza logstash[4365]: [2025-11-10T05:53:23,871][WARN ][logstash.outputs.elasticsearch][main] Attempted
Nov 10 05:53:28 hamza logstash[4365]: [2025-11-10T05:53:28,888][INFO ][logstash.outputs.elasticsearch][main] Failed to
Nov 10 05:53:28 hamza logstash[4365]: [2025-11-10T05:53:28,890][WARN ][logstash.outputs.elasticsearch][main] Attempted
Nov 10 05:53:34 hamza logstash[4365]: [2025-11-10T05:53:34,847][WARN ][logstash.outputs.elasticsearch][main] Restored c
Nov 10 05:53:35 hamza logstash[4365]: [2025-11-10T05:53:35,164][INFO ][logstash.outputs.elasticsearch][main] Elasticsea
Nov 10 05:53:35 hamza logstash[4365]: [2025-11-10T05:53:35,169][WARN ][logstash.outputs.elasticsearch][main] Detected a
Nov 10 05:54:21 hamza logstash[4365]: [2025-11-10T05:54:21,216][INFO ][logstash.outputs.elasticsearch][main] Using a de
lines 1-20/20 (END)
```

```
hamza@hamza:~$ sudo systemctl status kibana
● kibana.service - Kibana
   Loaded: loaded (/etc/systemd/system/kibana.service; disabled; preset: enabled)
   Active: active (running) since Mon 2025-11-10 05:45:22 CET; 15min ago
     Docs: https://www.elastic.co
   Main PID: 4324 (node)
    Tasks: 11 (limit: 10477)
  Memory: 362.6M (peak: 516.6M)
     CPU: 1min 25.162s
   CGroup: /system.slice/kibana.service
           └─4324 /usr/share/kibana/bin/../node/bin/node /usr/share/kibana/bin/../src/cli/dist --logging.dest=/var/log>

Nov 10 05:45:22 hamza systemd[1]: Started kibana.service - Kibana.
Nov 10 05:45:23 hamza kibana[4324]: Kibana is currently running with legacy OpenSSL providers enabled! For details and >
lines 1-13/13 (END)
```

All the services are “Active”

6. Started Logstash with the New Configuration

To ensure it reads new entries and sends them to Elasticsearch):

```
sudo service logstash start
```

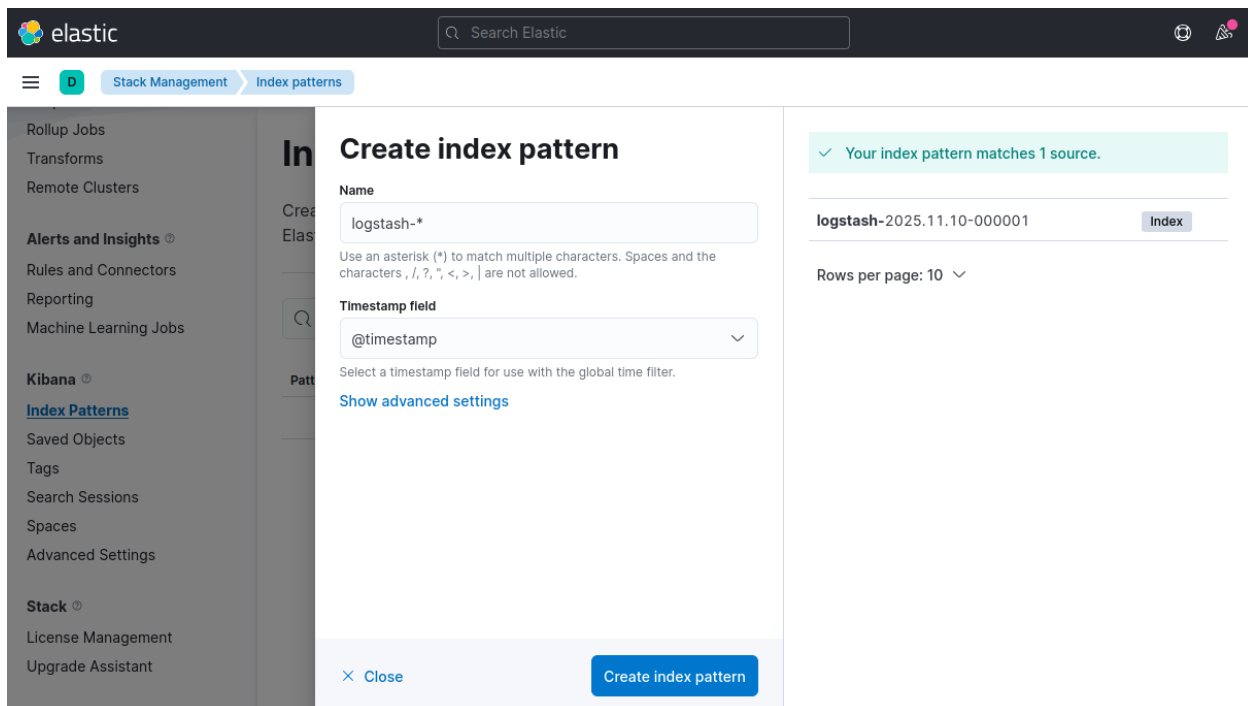
```
sudo /usr/share/logstash/bin/logstash -f /etc/logstash/conf.d/apache.conf
```

7. Visualized web traffic in a Kibana dashboard:

Opened kibana by using <http://localhost:5601>

Clicked on “Add integration” and then click on the menu “Stack Management” and then in Kibana section clicked on “Index Pattern”

Created new index pattern for logstash-.*.



Index of / x Final-project (1).pdf x logstash-* - Elastic x Certificate for hamza.lc x (no subject) - hamza x + v - o x

http://localhost:5601/app/management/kibana/indexPatterns/patterns/413a93c0-bddf-11

Files elastic Search Elastic Sign in

Stack Management Index patterns logstash-*

Rollup Jobs
Transforms
Remote Clusters

Alerts and Insights
Rules and Connectors
Reporting
Machine Learning Jobs

Kibana
Index Patterns
Saved Objects
Tags
Search Sessions
Spaces
Advanced Settings

Stack
License Management
Upgrade Assistant

logstash-*

Time field: '@timestamp'

View and edit fields in **logstash-***. Field attributes, such as type and searchability, are based on [field mappings](#) in Elasticsearch.

Fields (11) Scripted fields (0) Field filters (0)

Search All field types Add field

Name ↑	Type	Format	Searchable	Aggregatable	Excluded
@timestamp	date		•	•	
@version	keyword		•	•	
_id	_id		•	•	
_index	_index		•	•	
_score					
_source	_source				

After that we can see the logs in the Analytics and discovered to view the ingested logs.

localhost:5601/app/discover#/?_g=(filters:!(),refreshInterval:(pause:!t,value:0),time:(from:now))

elastic Search Elastic

Discover Options New Open Share Inspect Save

Search KQL Last 1 year Show dates Refresh

+ Add filter

logstash-*

Search field names

Filter by type 0

Available fields 21

- _id
- _index
- _score
- _type
- @timestamp
- @version
- agent
- auth
- bytes
- clientip
- host

13 hits

Chart options

Time ↓ Document

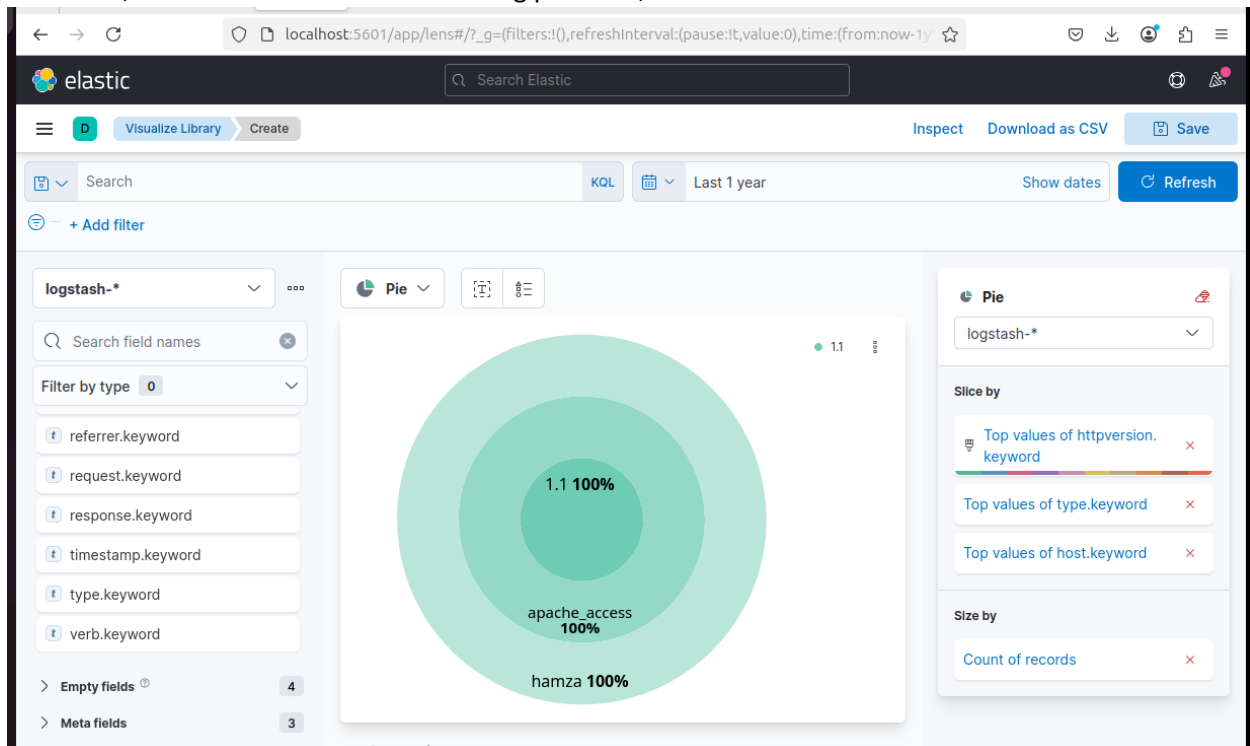
Nov 11, 2025 @ 02:12:39.000

```
@timestamp: Nov 11, 2025 @ 02:12:39.000 @version: 1 agent: "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:135.0) Gecko/20100101 Firefox/135.0" auth: - bytes: 581 clientip: 127.0.0.1 host: hamza httpversion: 1.1 ident: - message: 127.0.0.1 - - [11/Nov/2025:02:12:39 +0100] "GET / HTTP/1.1" 200 581 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:135.0) Gecko/20100101 Firefox/135.0" path: /var/log/apache2/
```

Nov 11, 2025 @ 02:12:38.000

```
@timestamp: Nov 11, 2025 @ 02:12:38.000 @version: 1 agent: "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:135.0) Gecko/20100101 Firefox/135.0" auth: - bytes: 432 clientip: 127.0.0.1 host: hamza httpversion: 1.1 ident: - message: 127.0.0.1 - - [11/Nov/2025:02:12:38 +0100] "GET /icons/blank.gif HTTP/1.1" 200 432 "http://
```

After that, created a visualization of it using pie chart,



And saved the visualization on the dashboard.

The 'Save dashboard' dialog box is shown. It has a title field with the text 'hamza.local', a description field, a tags field, and a checkbox labeled 'Store time with dashboard' which is checked. At the bottom, there are 'Cancel' and 'Save' buttons.

Save Lens visualization

Title

hamza.local

Description

Add to dashboard

☐ Existing

Search dashboards...

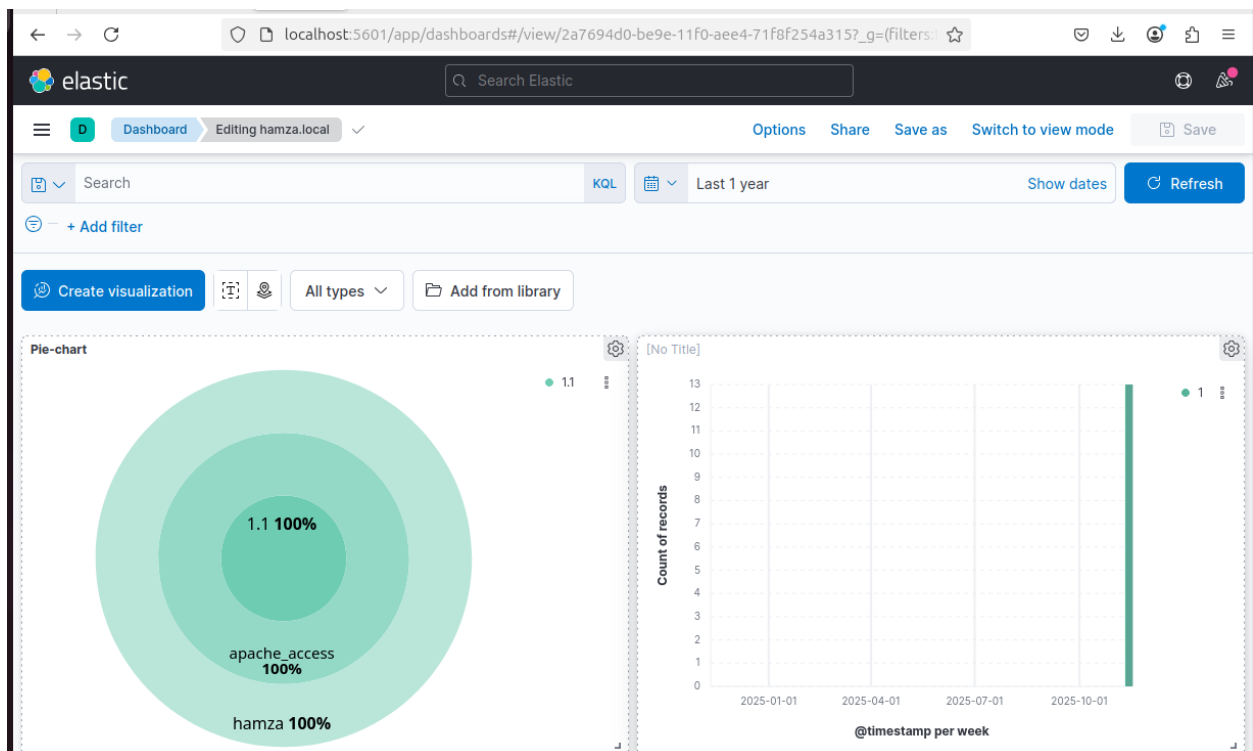
☒ New

☐ None

☐ Add to library ⓘ

Cancel

Save and go to Dashboard



3. Integrating Squid Proxy with ELK:

Purpose: Establish a proxy server and monitor its activity via ELK.

Steps:

1. Install and setup Squid:

1.1 Install squid:

```
sudo apt install squid
```

```
sudo systemctl status squid
```

```
hamza@hamza:~$ sudo apt install squid
[Output removed for brevity]

hamza@hamza:~$ sudo systemctl status squid
● squid.service - Squid Web Proxy Server
   Loaded: loaded (/usr/lib/systemd/system/squid.service; enabled; preset: enabled)
   Active: active (running) since Tue 2025-11-11 02:37:05 CET; 53s ago
     Docs: man:squid(8)
   Process: 12450 ExecStartPre=/usr/sbin/squid --foreground -z (code=exited, status=0/SUCCESS)
  Main PID: 12454 (squid)
    Tasks: 4 (limit: 4545)
   Memory: 18.0M (peak: 18.8M)
      CPU: 745ms
   CGroup: /system.slice/squid.service
           └─12454 /usr/sbin/squid --foreground -sYC
             └─12457 "(squid-1)" --kid squid-1 --foreground -sYC
               └─12458 "(logfile-daemon)" /var/log/squid/access.log
                 └─12459 "(pinger)"

Nov 11 02:37:05 hamza squid[12457]: Using Least Load store dir selection
Nov 11 02:37:05 hamza squid[12457]: Set Current Directory to /var/spool/squid
Nov 11 02:37:05 hamza squid[12457]: Finished loading MIME types and icons.
Nov 11 02:37:05 hamza squid[12457]: HTCP Disabled.
Nov 11 02:37:05 hamza squid[12457]: Pinger socket opened on FD 14
Nov 11 02:37:05 hamza squid[12457]: Squid plugin modules loaded: 0
Nov 11 02:37:05 hamza squid[12457]: Adaptation support is off.
Nov 11 02:37:05 hamza squid[12457]: Accepting HTTP Socket connections at conn3 local=[::]:3128 remote=[::] FD 12 flags=9
                                listening port: 3128
Nov 11 02:37:05 hamza systemd[1]: Started squid.service - Squid Web Proxy Server.
Nov 11 02:37:06 hamza squid[12457]: storeLateRelease: released 0 objects
```

1.2 Configure Squid to allow to allow traffic from the local network & the

localhost, and set the port Squid listens on: `sudo vim /etc/squid/squid.conf`

```
hamza@hamza:~$ sudo vim /etc/squid/squid.conf

http_access allow localnet
http_access allow localhost
spoof_client_ip allow|deny [!]aclname ...
#
#
#   If there are no "spoof_client_ip" lines present, the default
#   is to "allow" spoofing of any suitable request.
#
#   Note that the cache_peer "no-tproxy" option overrides this ACL.

1551,27-34
```

```
#           Request a client certificate during the TLS
#           handshake, but ignore certificate absence in
#           the TLS client Hello. If the client does
#           supply a certificate, it is validated.
#           NO_SESSION_REUSE
#           Don't allow for session reuse. Each connection
#           will result in a new SSL session.
#           VERIFY_CRL
#           Verify CRL lists when accepting client
#           certificates.
#           VERIFY_CRL_ALL
#           Verify CRL lists for all certificates in the
#           client certificate chain.
#
#           tls-default-ca[=off]
"/etc/squid/squid.conf" 9400L, 353462B 2106,16-24 22%
```

1.3 Restarted the Squid to apply changes: `sudo systemctl restart squid`

```
hamza@hamza:~$ sudo systemctl restart logstash
hamza@hamza:~$ sudo systemctl restart squid
```

2. Integration with ELK:

2.1 Create new Logstash pipeline to Process Squid Logs: `sudo vim /etc/logstash/conf.d/squid.conf`

```
input {
  file {
    path => "/var/log/squid/access.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
    type => "squid_log"
  }
}
filter {
  if [type] == "squid_log" {
    grok {
      match => { "message" => "%{NUMBER:timestamp} %{NUMBER}
%{INT:response_time} %{IP:src_ip}
%{WORD:squid_request_status} %{NUMBER:http_status_code}
%{NUMBER:reply_size} %{WORD:http_method} %{URI:requested_url}
%{USERNAME:user} %{WORD:squid_hierarchy_status} %{IP:dst_ip}" }
    }
    date {
      match => [ "timestamp", "UNIX" ]
    }
  }
}
output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "squid-logs-%{+YYYY.MM.dd}"
  }
}
-- INSERT --
```

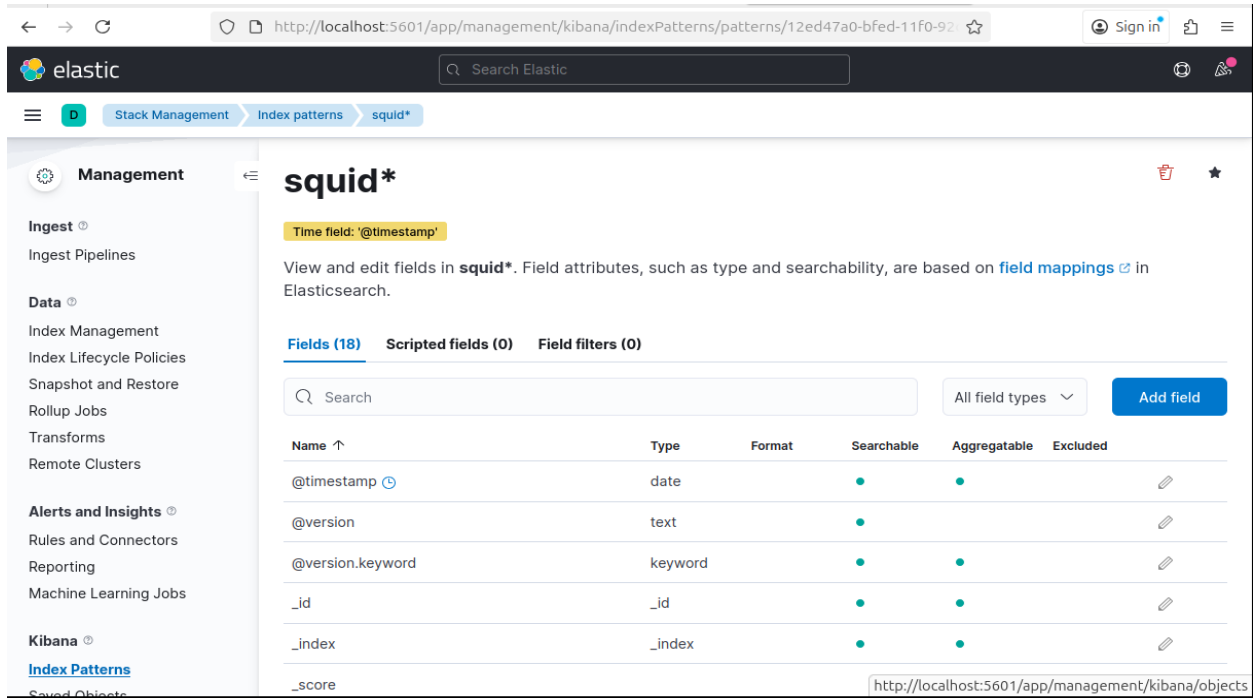
The input section tells Logstash to look at the Squid access log. The filter section processes the log lines using the grok plugin to match Squid's default log format. The output sends the processed logs to Elasticsearch.

2.2 Reload/Restart Logstash: `sudo systemctl restart logstash`

3. Visualized Squid Logs in Kibana:

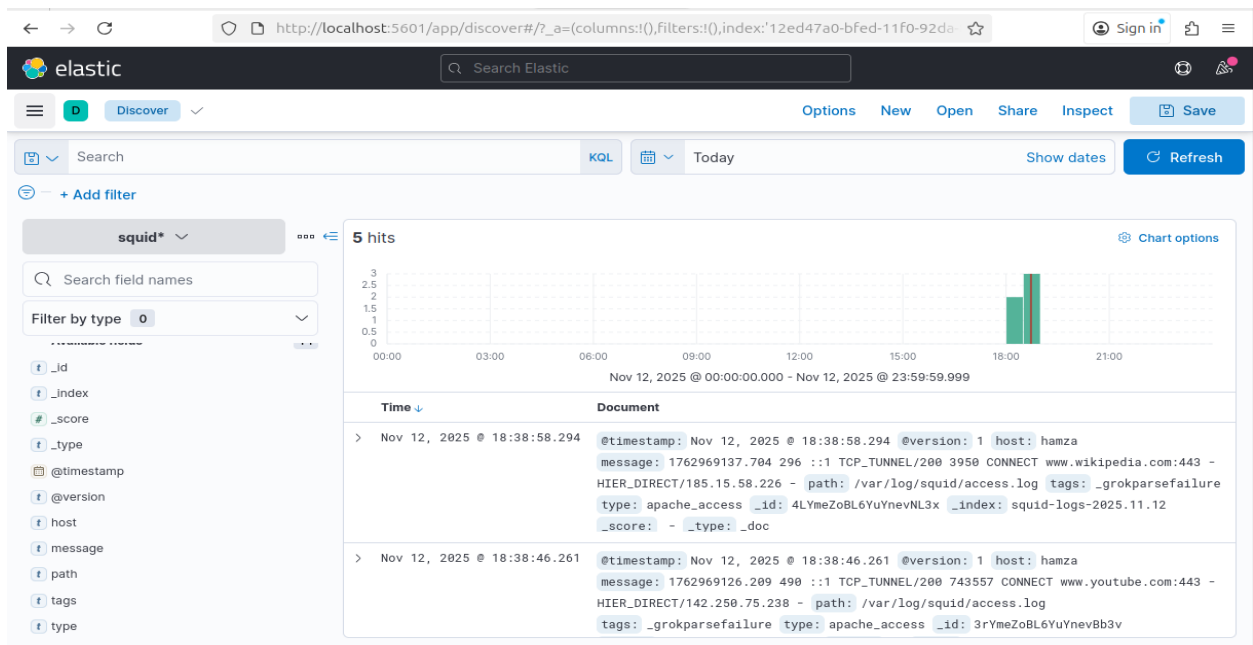
3.1 Opened Kibana in a browser: <http://localhost:5601>

3.2 Navigated to "Discover" to view the Squid logs that have been ingested. Stack management: Index patterns: Index pattern "squid-*", Timestamp field "@timestamp"



The screenshot shows the Kibana Stack Management interface for the index pattern 'squid*'. The left sidebar contains navigation links for Management, Ingest, Data, Alerts and Insights, and Kibana. The main content area displays the 'squid*' index pattern with a time field set to '@timestamp'. Below this, there is a table of fields (18) with columns for Name, Type, Format, Searchable, Aggregatable, and Excluded. The fields listed are @timestamp (date), @version (text), @version.keyword (keyword), _id (_id), _index (_index), and _score (_score). A search bar and an 'Add field' button are also visible.

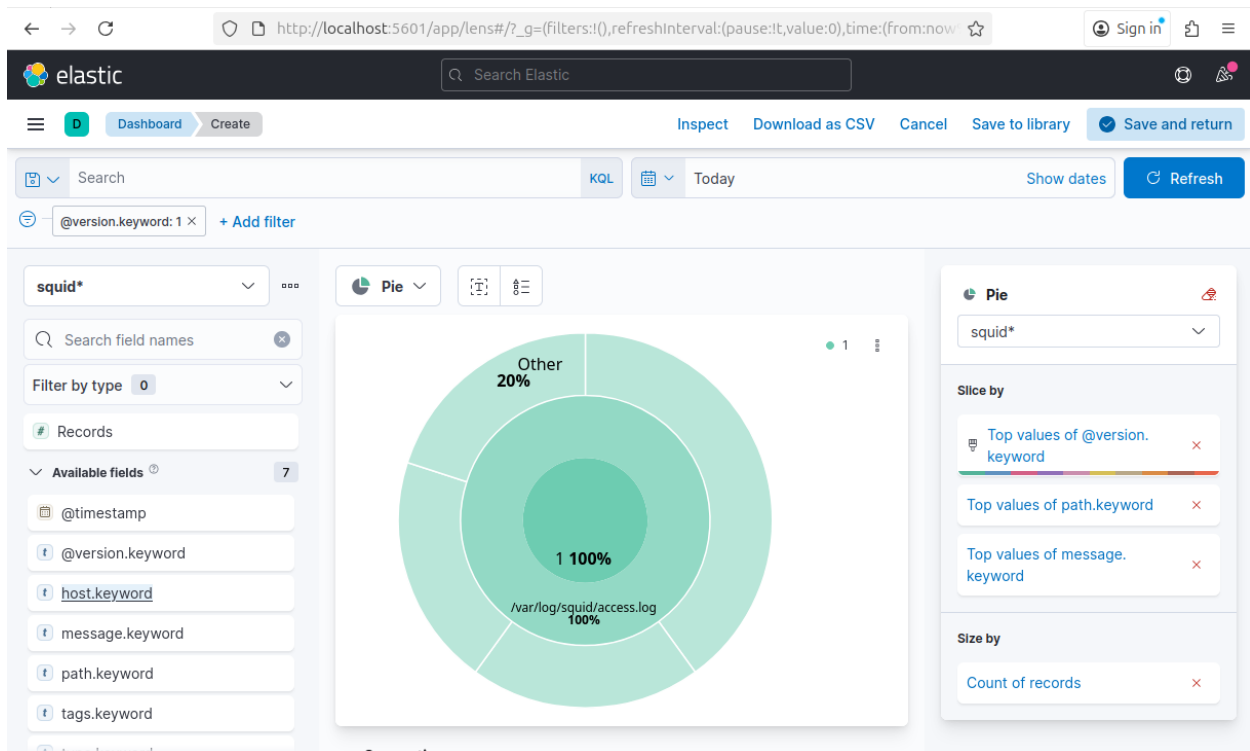
Name	Type	Format	Searchable	Aggregatable	Excluded
@timestamp	date		●	●	
@version	text		●		
@version.keyword	keyword		●	●	
_id	_id		●	●	
_index	_index		●	●	
_score					



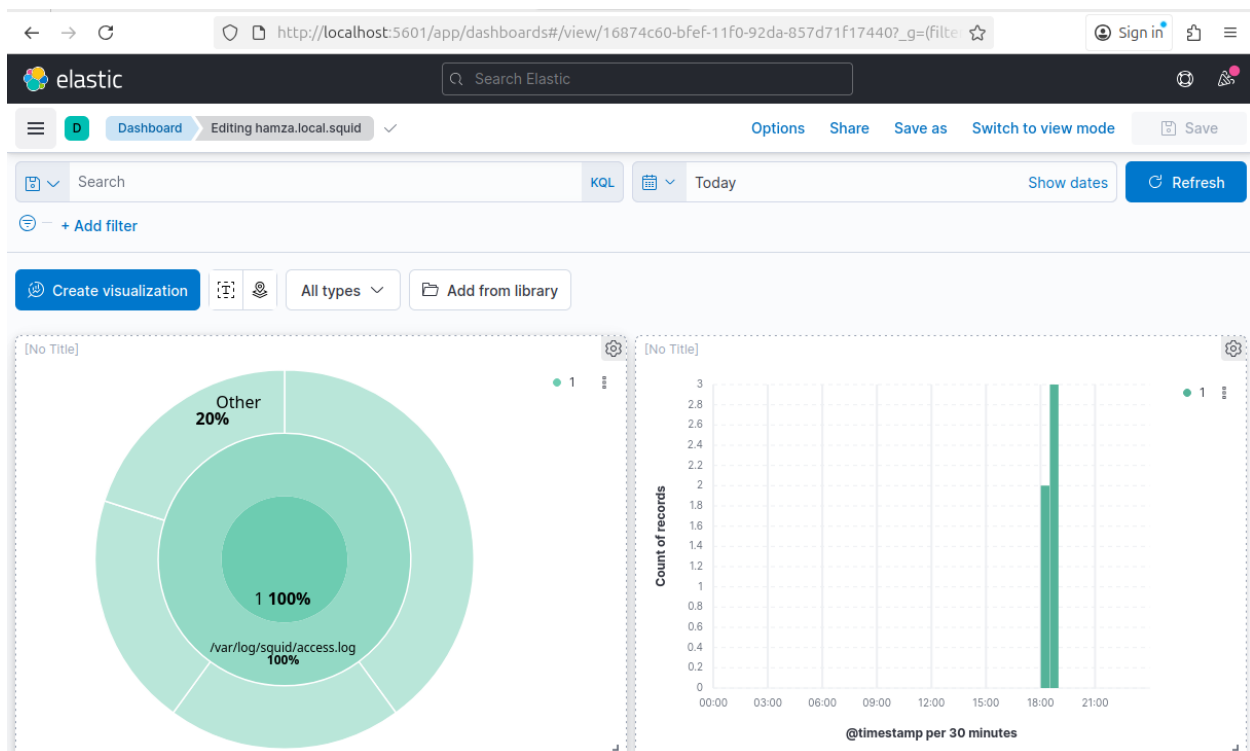
The screenshot shows the Kibana Discover interface for the 'squid*' index pattern. The left sidebar contains a search bar and a list of fields to filter by. The main content area displays a search bar, a KQL query editor, and a 'Today' button. Below this, there is a chart showing 5 hits. The chart is a bar chart with a time axis from 00:00 to 21:00. The hits are displayed in a table with columns for Time and Document. The first hit is from Nov 12, 2025 @ 18:38:58.294 and the second is from Nov 12, 2025 @ 18:38:46.261. Both hits are of type 'apache_access' and have a '_score' of -1.

Time	Document
Nov 12, 2025 @ 18:38:58.294	<pre>@timestamp: Nov 12, 2025 @ 18:38:58.294 @version: 1 host: hamza message: 1762969137.704 296 ::1 TCP_TUNNEL/200 3950 CONNECT www.wikipedia.com:443 - HIER_DIRECT/185.15.58.226 - path: /var/log/squid/access.log tags: _grokparsefailure type: apache_access _id: 4LYmeZoBL6YuYnevNL3x _index: squid-logs-2025.11.12 _score: -1 _type: _doc</pre>
Nov 12, 2025 @ 18:38:46.261	<pre>@timestamp: Nov 12, 2025 @ 18:38:46.261 @version: 1 host: hamza message: 1762969126.209 490 ::1 TCP_TUNNEL/200 743557 CONNECT www.youtube.com:443 - HIER_DIRECT/142.250.75.238 - path: /var/log/squid/access.log tags: _grokparsefailure type: apache_access _id: 3rYmeZoBL6YuYnevBb3v</pre>

3.3 Created relevant visualizations such as pie charts



3.4 Added these visualizations to a dedicated dashboard or integrate them with the web server's dashboard for a consolidated view.



4. DMZ Setup:

Purpose: Structure entire setup to segment for ELK and Squid.

Steps:

1. Setup iptables:

```
sudo apt-get update
```

```
sudo apt-get install iptables
```

```
hamza@hamza:~$ sudo apt-get update
```

```
hamza@hamza:~$ sudo apt-get install iptables
```

2. Flushing existing rules and set default policy to drop (to ensure a clean state):

```
sudo iptables -F
```

```
sudo iptables -P INPUT DROP
```

```
sudo iptables -P FORWARD DROP
```

```
sudo iptables -P OUTPUT DROP
```

```
hamza@hamza:~$ sudo iptables -F
hamza@hamza:~$ sudo iptables -P INPUT DROP
hamza@hamza:~$ sudo iptables -P FORWARD DROP
hamza@hamza:~$ sudo iptables -P OUTPUT DROP
```

3. Allowing Traffic on Loopback Interface (to enable traffic for services running there):

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

```
sudo iptables -A OUTPUT -o lo -j ACCEPT
```

```
hamza@hamza:~$ sudo iptables -A INPUT -i lo -j ACCEPT
hamza@hamza:~$ sudo iptables -A OUTPUT -o lo -j ACCEPT
```

4. Allowing Outgoing Traffic on the External Interface:

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

```
hamza@hamza:~$ sudo iptables -A INPUT -i lo -j ACCEPT
```

No, it is not necessary loopback traffic is already permitted. To allow outgoing traffic, just add an OUTPUT rule. There are no specific ports to open, since outbound requests originate from the server itself.

5. Allowing Incoming HTTP and HTTPS Traffic on the External Interface

```
sudo iptables -A INPUT -i ens33 -p tcp --dport 80 -j ACCEPT
```

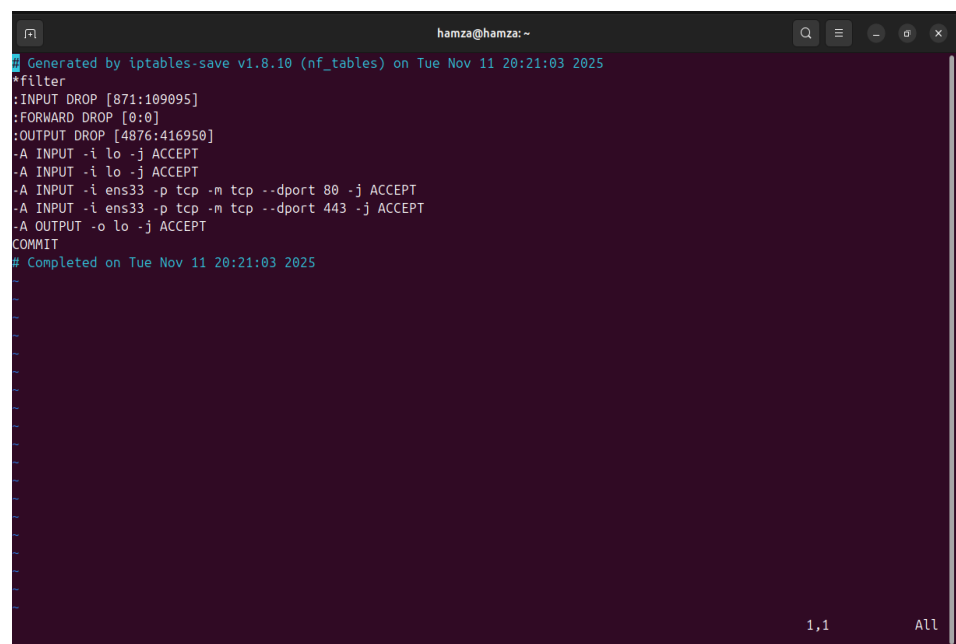
```
hamza@hamza:~$ sudo iptables -A INPUT -i ens33 -p tcp --dport 80 -j ACCEPT
hamza@hamza:~$ sudo iptables -A INPUT -i ens33 -p tcp --dport 443 -j ACCEPT
```

6. Saved the iptables Configuration (so that the rules persist after a reboot):

```
view /etc/iptables/rules.v4
```

```
hamza@hamza:~$ sudo iptables-save | sudo tee /etc/iptables/rules.v4
# Generated by iptables-save v1.8.10 (nf_tables) on Tue Nov 11 20:21:03 2025
*filter
:INPUT DROP [871:109095]
:FORWARD DROP [0:0]
:OUTPUT DROP [4876:416950]
-A INPUT -i lo -j ACCEPT
-A INPUT -i lo -j ACCEPT
-A INPUT -i ens33 -p tcp -m tcp --dport 80 -j ACCEPT
-A INPUT -i ens33 -p tcp -m tcp --dport 443 -j ACCEPT
-A OUTPUT -o lo -j ACCEPT
COMMIT
# Completed on Tue Nov 11 20:21:03 2025
```

```
hamza@hamza:~$ view /etc/iptables/rules.v4
```



Security Considerations:

The purpose of the Demilitarized Zone (DMZ) is to create a distinction between public services and internal networks to ensure as much security as possible by exposing only ports 80 and 443 to mitigate vulnerability risk. All Services which are using SSL/TLS through Apache to encrypt the communications, which makes the capturing data very difficult (including Kibana access). As important, access to Kibana is granted only to authorized internal users, and rigorous authentication and role-based access control (RBAC) can restrict internal users to only viewing data relevant to their respective role.

Future improvements or extensions:

I think these are following improvement or extensions which are given below:

- To enable the user authentication with role-based access in Kibana.
- To use Let's Encrypt for publicly trusted SSL/TLS.
- To deploy components to run in containers for scalability by using Docker.